1.

Python Activity-Based Learning

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

Sol:

Problem statement

Evaluate scholarship applicants based on aggregate marks, subject scores, attendance, and family income to determine eligibility, award tiers, and tie-breaking.

Objectives

Evaluate academic eligibility.

Apply financial income filters.

Determine scholarship award category.

Assumptions

Aggregate marks out of 100.

Minimum attendance required is 75%.

Annual income threshold is 250,000.

Algorithm

1. Read aggregate marks, attendance percentage, and annual income.

2. Validate inputs.

3. If marks >= 85, attendance >= 75%, income <= 250000, award Full Scholarship.

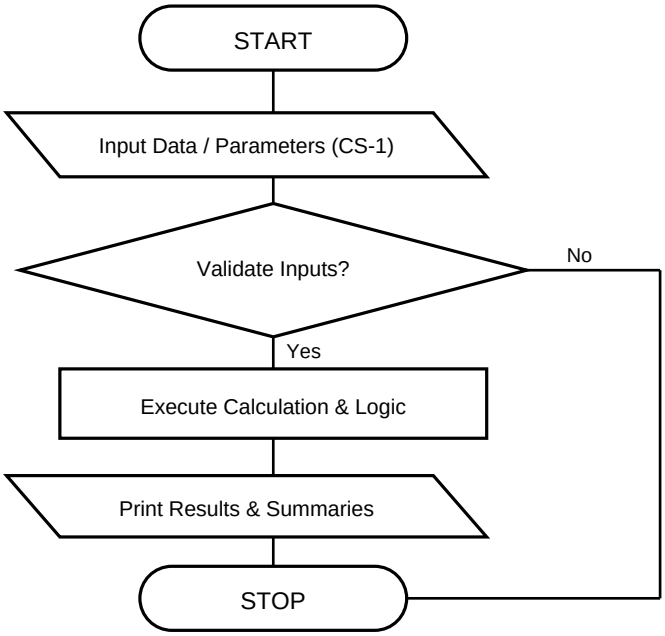
4. Else if marks >= 75 and attendance >= 75%, award Partial Scholarship.

5. Else mark as Not Eligible.

Pseudocode

```
START
INPUT marks, attendance, income
IF marks < 0 OR marks > 100 OR attendance < 0 OR attendance > 100 OR income <
 0 THEN
    PRINT "Invalid input"
ELSE IF marks >= 85 AND attendance >= 75 AND income <= 250000 THEN
    PRINT "Eligible: Full Scholarship"
ELSE IF marks >= 75 AND attendance >= 75 THEN
    PRINT "Eligible: Partial Scholarship"
ELSE
    PRINT "Not Eligible"
END IF
STOP
```

Flowchart



Source Code

```
marks = float(input("Enter aggregate marks (%): "))
attendance = float(input("Enter attendance (%): "))
income = float(input("Enter annual family income: "))

if marks < 0 or marks > 100 or attendance < 0 or attendance > 100 or income <
 0:
    print("Invalid input.")
elif marks >= 85 and attendance >= 75 and income <= 250000:
    print("Scholarship Status: Full Scholarship Granted")
elif marks >= 75 and attendance >= 75:
    print("Scholarship Status: Partial Scholarship Granted")
else:
    print("Scholarship Status: Not Eligible")
```

Test Cases

1. Marks: 90, Attendance: 80, Income: 200000 -> Output: Full Scholarship Granted

2. Marks: 78, Attendance: 80, Income: 300000 -> Output: Partial Scholarship Granted

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter aggregate marks (%): 90
Enter attendance (%): 80
Enter annual family income: 200000
Scholarship Status: Full Scholarship Granted

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter aggregate marks (%): 78
Enter attendance (%): 80
Enter annual family income: 300000
Scholarship Status: Partial Scholarship Granted

Process finished with exit code 0
```

Conclusion

The scholarship allocation engine successfully evaluates academic performance and financial criteria to classify candidates.

2.

Python Activity-Based Learning

Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

Sol:

Problem statement

Calculate household electricity bills using progressive consumption slabs and unit rates.

Objectives

Process unit consumption inputs.

Apply slab-based tariff rules.

Output total electricity charges.

Assumptions

First 100 units: 2 per unit.

Next 100 units: 3 per unit.

Units above 200: 5 per unit.

Algorithm

1. Read electricity units consumed.

2. Validate inputs.

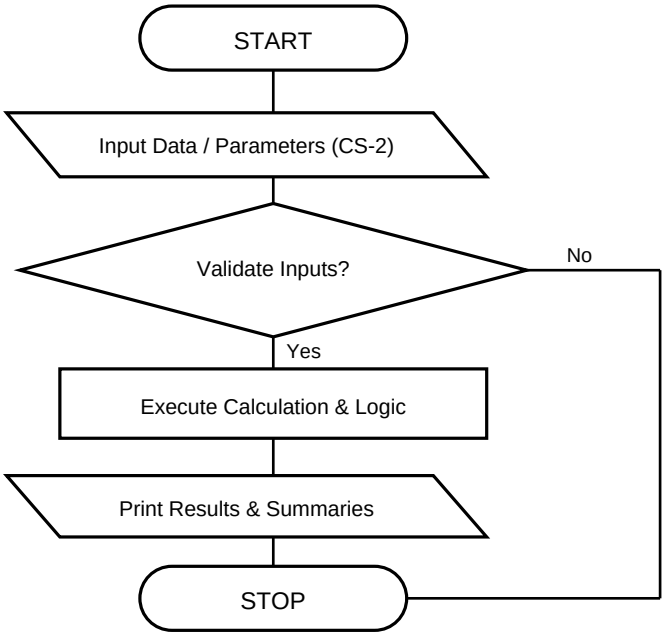
3. Calculate progressive bill based on usage slabs.

4. Output total charges.

Pseudocode

```
Start
Input units
if units < 0
    print "Invalid input"
else IF units <= 100
    bill = units * 2
ELSE IF units <= 200
    bill = 100 * 2 + (units - 100) * 3
ELSE
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
PRINT bill
STOP
```

Flowchart



Source Code

```
units = int(input("Enter electricity units: "))

if units < 0:
    print("Invalid input.")
elif units <= 100:
    bill = units * 2
    print("Electricity bill: Rs.", bill)
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
    print("Electricity bill: Rs.", bill)
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
    print("Electricity bill: Rs.", bill)
```

Test Cases

1. Enter electricity units: 80 -> Output: Electricity bill: Rs. 160
2. Enter electricity units: 250 -> Output: Electricity bill: Rs. 750

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\2.py
Enter electricity units: 80
Electricity bill: Rs. 160

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\2.py
Enter electricity units: 250
Electricity bill: Rs. 750

Process finished with exit code 0
```

Conclusion

The progressive tariff model successfully calculates consumer charges according to defined multi-tier rate brackets.

3.

Python Activity-Based Learning

Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

Sol:

Problem statement

Calculate a patient's hospital bill based on consultation, diagnostic, and treatment charges. Classify the bill amount and apply concessions.

Objectives

Calculate hospital bill based on services.

Classify bill into Basic, Standard, or High.

Apply 10% concession for bills >= 5,000.

Assumptions

Concession rate of 10% for total >= 5,000.

Basic (<2000), Standard (2000-5000), High (>5000).

Algorithm

1. Read consultation, diagnostic, and treatment charges.

2. Validate inputs.

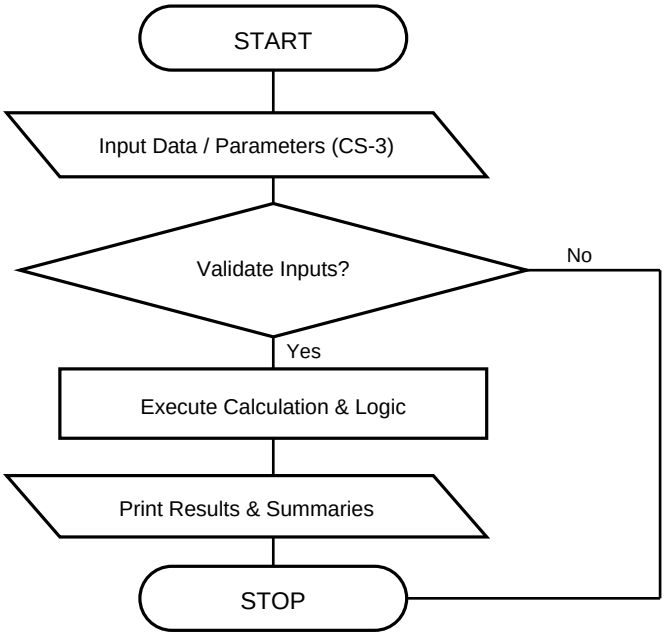
3. Calculate total and apply 10% concession if total >= 5000.

4. Classify patient into Basic, Standard, or High.

Pseudocode

```
START
INPUT consultation, diagnostic, treatment
IF any charge < 0 THEN
    PRINT "Invalid charges"
ELSE
    total = consultation + diagnostic + treatment
    IF total >= 5000 THEN total = total - total * 0.10
    IF total < 2000 THEN category = "Basic"
    ELSE IF total <= 5000 THEN category = "Standard"
    ELSE category = "High"
    PRINT total, category
END IF
STOP
```

Flowchart



Source Code

```
consult = float(input("Consultation charge: "))
tests = float(input("Diagnostic charges: "))
treatment = float(input("Treatment charges: "))

if consult < 0 or tests < 0 or treatment < 0:
    print("Invalid charges.")
else:
    total = consult + tests + treatment
    if total >= 5000:
        concession = total * 10 / 100
        total = total - concession
    if total < 2000:
        category = "Basic"
    elif total <= 5000:
        category = "Standard"
    else:
        category = "High"
    print("Final bill: Rs.", round(total, 2))
    print("Patient category:", category)
```

Test Cases

1. Consult: 500, Tests: 300, Treatment: 700 -> Final bill: Rs. 1500.0, Category: Basic

2. Consult: 2000, Tests: 1500, Treatment: 2500 -> Final bill: Rs. 5400.0, Category: High

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\3.py
Consultation charge: 500
Diagnostic charges: 300
Treatment charges: 700
Final bill: Rs. 1500.0
Patient category: Basic

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\3.py
Consultation charge: 2000
Diagnostic charges: 1500
```

```
Treatment charges: 2500
Final bill: Rs. 5400.0
Patient category: High

Process finished with exit code 0
```

Conclusion

The program calculates hospital bills, applies a concession when eligible, validates charges, and classifies patients.

4.

Python Activity-Based Learning

Employee Payroll and Deduction Analysis: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

Sol:

Problem statement

Calculate gross salary from basic pay, HRA, DA, and overtime, then apply progressive deductions (PF and Tax) to generate net salary.

Objectives

Compute gross salary components.

Calculate PF and progressive income tax.

Generate detailed net pay breakdown.

Assumptions

HRA = 20% of Basic, DA = 10% of Basic.

PF = 12% of Basic.

Tax = 10% if Gross > 50,000, else 5%.

Algorithm

1. Read basic pay and overtime pay.

2. Validate inputs.

3. HRA = 0.20 * basic, DA = 0.10 * basic.

4. Gross = basic + HRA + DA + overtime.

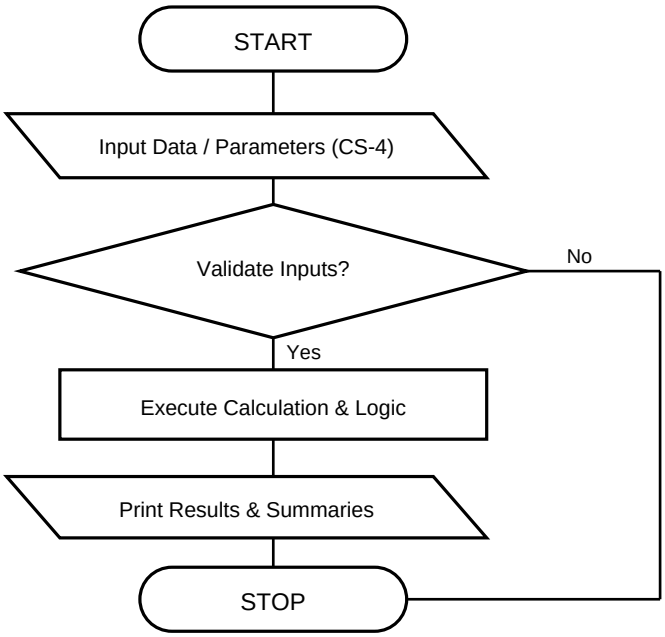
5. PF = 0.12 * basic; Tax = 0.10 * gross if gross > 50000 else 0.05 * gross.

6. Net = gross - (PF + Tax).

Pseudocode

```
START
INPUT basic, overtime
IF basic < 0 OR overtime < 0 THEN
    PRINT "Invalid input"
ELSE
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da + overtime
    pf = basic * 0.12
    IF gross > 50000 THEN tax = gross * 0.10 ELSE tax = gross * 0.05
    net = gross - (pf + tax)
    PRINT gross, pf, tax, net
END IF
STOP
```

Flowchart



Source Code

```
basic = float(input("Enter basic pay: "))
overtime = float(input("Enter overtime pay: "))

if basic < 0 or overtime < 0:
    print("Invalid input.")
else:
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da + overtime
    pf = basic * 0.12
    tax = gross * 0.10 if gross > 50000 else gross * 0.05
    net = gross - (pf + tax)
    print("Gross Salary: Rs.", round(gross, 2))
    print("Total Deductions (PF+Tax): Rs.", round(pf + tax, 2))
    print("Net Salary: Rs.", round(net, 2))
```

Test Cases

1. Basic: 40000, Overtime: 2000 -> Gross: 54000.0, Deductions: 10200.0, Net: 43800.0

2. Basic: 20000, Overtime: 0 -> Gross: 26000.0, Deductions: 3700.0, Net: 22300.0

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\4.py
Enter basic pay: 40000
Enter overtime pay: 2000
Gross Salary: Rs. 54000.0
Total Deductions (PF+Tax): Rs. 10200.0
Net Salary: Rs. 43800.0

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\4.py
Enter basic pay: 20000
Enter overtime pay: 0
Gross Salary: Rs. 26000.0
Total Deductions (PF+Tax): Rs. 3700.0
Net Salary: Rs. 22300.0

Process finished with exit code 0
```

Conclusion

The payroll module accurately computes salary allowances, progressive taxes, and statutory deductions.

5.

Python Activity-Based Learning

Multi-Plan Telecommunications Cost Optimization: Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

Sol:

Problem statement

Compare standard vs premium mobile telecommunication plans based on data and call usage to recommend the cheapest plan.

Objectives

Compute total charges for multiple telecom plans.

Factor in data overage fees.

Recommend the most economical plan.

Assumptions

Plan A: Rs 299 (includes 30GB, excess Rs 10/GB).

Plan B: Rs 499 (includes 75GB, excess Rs 5/GB).

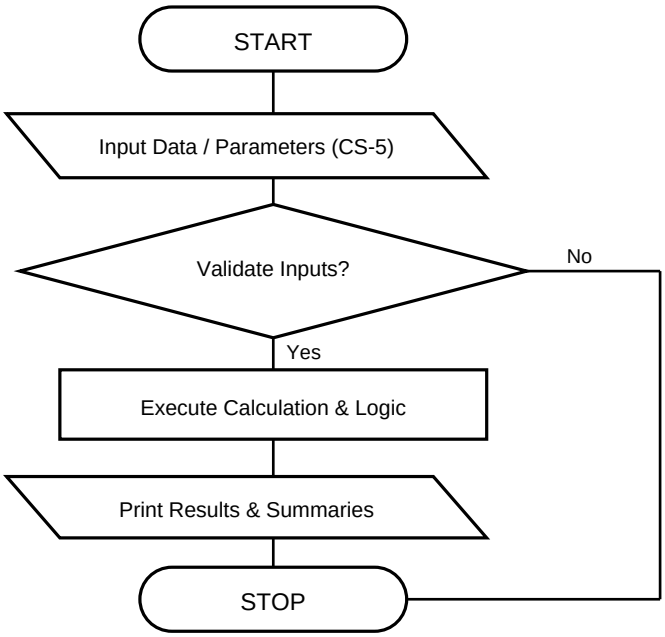
Algorithm

1. Read data usage in GB.
2. Validate input.
3. Calculate cost for Plan A = 299 + max(0, usage - 30) * 10.
4. Calculate cost for Plan B = 499 + max(0, usage - 75) * 5.
5. Compare costs and select the cheapest plan.

Pseudocode

```
START
INPUT usage
IF usage < 0 THEN
    PRINT "Invalid usage"
ELSE
    costA = 299 + MAX(0, usage - 30) * 10
    costB = 499 + MAX(0, usage - 75) * 5
    IF costA <= costB THEN
        PRINT "Plan A", costA
    ELSE
        PRINT "Plan B", costB
    END IF
END IF
STOP
```

Flowchart



Source Code

```
usage = float(input("Enter monthly data usage (GB): "))

if usage < 0:
    print("Invalid usage.")
else:
    extraA = max(0.0, usage - 30)
    costA = 299 + extraA * 10
    extraB = max(0.0, usage - 75)
    costB = 499 + extraB * 5
    print(f"Plan A Cost: Rs. {costA:.2f}")
    print(f"Plan B Cost: Rs. {costB:.2f}")
    if costA <= costB:
        print("Recommended Plan: Plan A")
    else:
        print("Recommended Plan: Plan B")
```

Test Cases

1. Usage: 40 GB -> Plan A: 399.0, Plan B: 499.0 -> Recommended: Plan A

2. Usage: 90 GB -> Plan A: 899.0, Plan B: 574.0 -> Recommended: Plan B

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\5.py
Enter monthly data usage (GB): 40
Plan A Cost: Rs. 399.00
Plan B Cost: Rs. 499.00
Recommended Plan: Plan A

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\5.py
Enter monthly data usage (GB): 90
Plan A Cost: Rs. 899.00
Plan B Cost: Rs. 574.00
Recommended Plan: Plan B

Process finished with exit code 0
```

Conclusion

The optimization system successfully selects the most cost-effective mobile plan according to consumer usage.

6.

Python Activity-Based Learning

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

Sol:

Problem statement

Calculate passenger fares using distance, travel class, age concessions, reservation fees and cancellation penalties.

Objectives

Calculate distance-based fares.

Apply class charges and age concessions.

Calculate reservation and cancellation charges.

Assumptions

Base fare is 2 per kilometre.

Class multipliers: Sleeper (1x), AC (2x).

Age >= 60 gets 20% concession.

Reservation fee: Rs 50.

Cancellation penalty: 10% of concession fare.

Algorithm

1. Input distance, age, and travel class.

2. Calculate base fare = distance * 2.

3. If class is AC, multiply fare by 2.

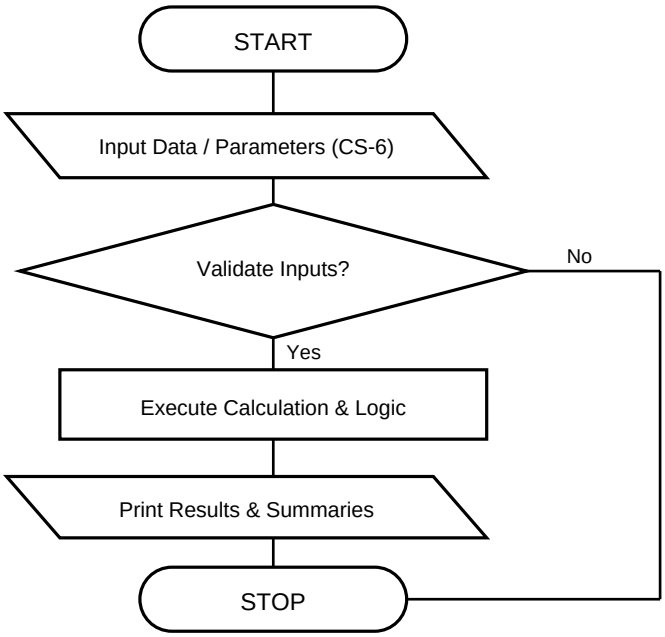
4. If age >= 60, apply 20% discount.

5. Add Rs 50 reservation fee; calculate cancellation penalty as 10% of fare.

Pseudocode

```
START
INPUT distance, age, class
CALCULATE base fare
APPLY class multiplier
IF age >= 60 THEN APPLY concession
ADD reservation fee
CALCULATE cancellation penalty
DISPLAY fare and penalty
STOP
```

Flowchart



Source Code

```
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter Sleeper or AC: ").lower()

if distance <= 0 or age < 0:
    print("Invalid input")
elif travel_class not in ["sleeper", "ac"]:
    print("Invalid class")
else:
    fare = distance * 2
    if travel_class == "ac":
        fare = fare * 2
    if age >= 60:
        fare = fare * 0.80
    total = fare + 50
    penalty = fare * 0.10
    print("Fare after concession:", fare)
    print("Total fare:", total)
    print("Cancellation penalty:", penalty)
```

Test Cases

1. Distance: 200, Age: 25, Class: Sleeper -> Fare: 400.0, Total: 450.0, Penalty: 40.0

2. Distance: 300, Age: 65, Class: AC -> Fare: 960.0, Total: 1010.0, Penalty: 96.0

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\6.py
Enter distance in km: 200
Enter passenger age: 25
Enter Sleeper or AC: sleeper
Fare after concession: 400.0
Total fare: 450.0
Cancellation penalty: 40.0

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\6.py
Enter distance in km: 300
Enter passenger age: 65
Enter Sleeper or AC: ac
Fare after concession: 960.0
Total fare: 1010.0
Cancellation penalty: 96.0

Process finished with exit code 0
```

Conclusion

The program calculates railway fares and cancellation penalties according to stated rules.

7.

Python Activity-Based Learning

Retail Billing with Multiple Discount Rules: Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

Sol:

Problem statement

Calculate retail invoice total across multiple items, applying category taxes and bulk purchase discounts.

Objectives

Process product prices and quantities.

Apply 5% store discount if total > 2000.

Calculate 12% GST tax on final bill.

Assumptions

Discount threshold: Rs 2000.

GST rate: 12%.

Algorithm

1. Read item price and quantity.

2. Validate inputs.

3. Subtotal = price * quantity.

4. If subtotal > 2000, discount = 0.05 * subtotal.

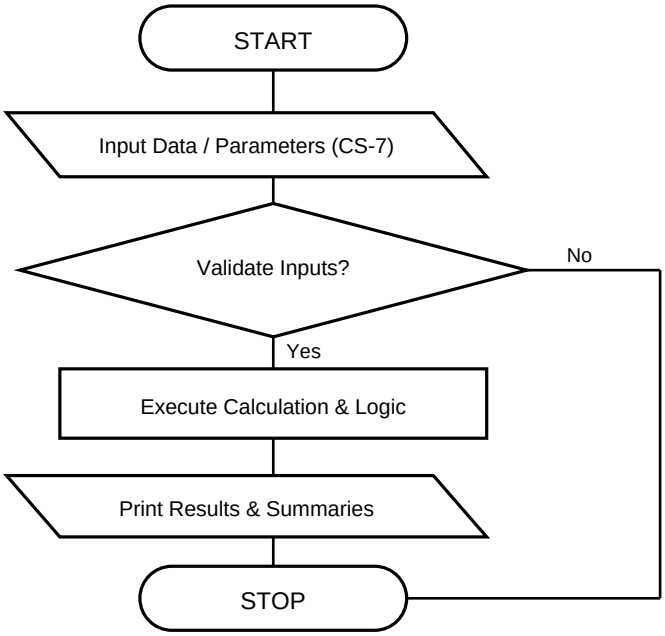
5. Tax = 0.12 * (subtotal - discount).

6. Total = subtotal - discount + tax.

Pseudocode

```
START
INPUT price, quantity
IF price <= 0 OR quantity <= 0 THEN
    PRINT "Invalid input"
ELSE
    subtotal = price * quantity
    IF subtotal > 2000 THEN discount = subtotal * 0.05 ELSE discount = 0
    tax = (subtotal - discount) * 0.12
    total = subtotal - discount + tax
    PRINT subtotal, discount, tax, total
END IF
STOP
```

Flowchart



Source Code

```
price = float(input("Enter unit price: "))
qty = int(input("Enter quantity: "))

if price <= 0 or qty <= 0:
    print("Invalid input.")
else:
    subtotal = price * qty
    discount = subtotal * 0.05 if subtotal > 2000 else 0.0
    taxable = subtotal - discount
    tax = taxable * 0.12
    final_total = taxable + tax
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("GST (12%): Rs.", round(tax, 2))
    print("Final Invoice Total: Rs.", round(final_total, 2))
```

Test Cases

1. Price: 500, Qty: 5 -> Subtotal: 2500.0, Disc: 125.0, Tax: 285.0, Total: 2660.0

2. Price: 100, Qty: 10 -> Subtotal: 1000.0, Disc: 0.0, Tax: 120.0, Total: 1120.0

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\7.py
Enter unit price: 500
Enter quantity: 5
Subtotal: Rs. 2500.0
Discount: Rs. 125.0
GST (12%): Rs. 285.0
Final Invoice Total: Rs. 2660.0

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\7.py
Enter unit price: 100
Enter quantity: 10
Subtotal: Rs. 1000.0
Discount: Rs. 0.0
GST (12%): Rs. 120.0
Final Invoice Total: Rs. 1120.0

Process finished with exit code 0
```

Conclusion

The retail billing module accurately processes product sub-totals, discounts, and GST taxes.

8.

Python Activity-Based Learning

Manufacturing Cost and Profitability Analysis: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

Sol:

Problem statement

Calculate total manufacturing costs and determine whether selling price meets a target profit margin.

Objectives

Compute production cost components.

Calculate profit or loss percentage.

Evaluate target profit feasibility.

Assumptions

Total Cost = Material + Labour + Machine Overhead.

Profit Margin % = ((Selling Price - Total Cost) / Total Cost) * 100.

Algorithm

1. Read material cost, labour cost, overhead cost, and selling price.

2. Validate inputs.

3. Total cost = material + labour + overhead.

4. Profit = selling price - total cost.

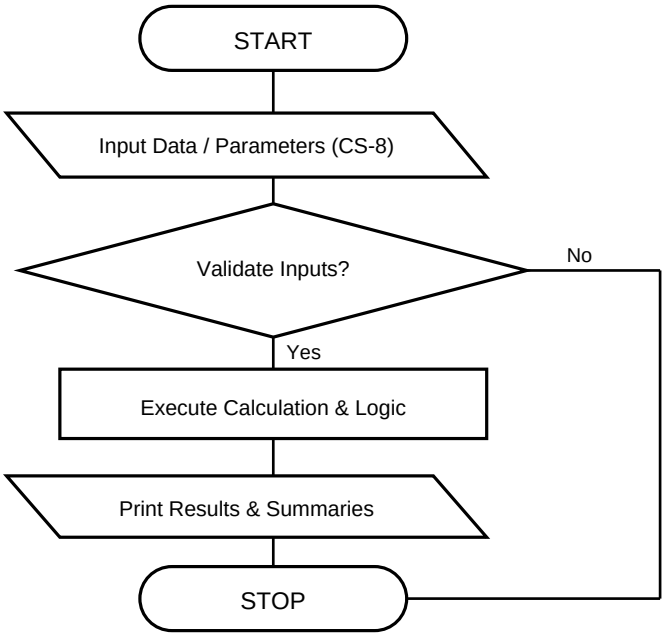
5. Profit % = (profit / total cost) * 100.

6. Display profit status.

Pseudocode

```
START
INPUT material, labour, overhead, selling_price
IF material < 0 OR labour < 0 OR overhead < 0 OR selling_price < 0 THEN
    PRINT "Invalid input"
ELSE
    total_cost = material + labour + overhead
    profit = selling_price - total_cost
    profit_percent = (profit / total_cost) * 100
    PRINT total_cost, profit, profit_percent
END IF
STOP
```

Flowchart



Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
overhead = float(input("Enter machine overhead cost: "))
selling_price = float(input("Enter expected selling price: "))

if material < 0 or labour < 0 or overhead < 0 or selling_price < 0:
    print("Invalid cost inputs.")
else:
    total_cost = material + labour + overhead
    profit = selling_price - total_cost
    profit_pct = (profit / total_cost) * 100 if total_cost > 0 else 0
    print("Total Manufacturing Cost: Rs.", round(total_cost, 2))
    print("Net Profit/Loss: Rs.", round(profit, 2))
    print("Profit Margin:", round(profit_pct, 2), "%")
    if profit_pct >= 15:
        print("Status: Production Target Met (Profitable)")
    else:
        print("Status: Target Not Met (Low Margin)")
```

Test Cases

1. Material: 500, Labour: 300, Overhead: 200, Selling Price: 1300 -> Cost: 1000, Profit: 300 (30.0%)

2. Material: 500, Labour: 300, Overhead: 200, Selling Price: 1050 -> Cost: 1000, Profit: 50 (5.0%)

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\8.py
Enter material cost: 500
Enter labour cost: 300
Enter machine overhead cost: 200
Enter expected selling price: 1300
Total Manufacturing Cost: Rs. 1000.0
Net Profit/Loss: Rs. 300.0
Profit Margin: 30.0 %
Status: Production Target Met (Profitable)

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\8.py
Enter material cost: 500
Enter labour cost: 300
Enter machine overhead cost: 200
Enter expected selling price: 1050
Total Manufacturing Cost: Rs. 1000.0
Net Profit/Loss: Rs. 50.0
Profit Margin: 5.0 %
Status: Target Not Met (Low Margin)

Process finished with exit code 0
```

Conclusion

The profitability analysis program evaluates total manufacturing costs against target selling prices.

9.

Python Activity-Based Learning

Multi-Currency Travel Budget Analysis: Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

Sol:

Problem statement

Convert foreign travel expenses to local currency, include conversion fees, and check against total travel budget.

Objectives

Convert foreign currency amounts to INR.

Apply 2% currency conversion fee.

Compare total spending against budget.

Assumptions

1 USD = 83 INR.

Conversion fee = 2% of total INR.

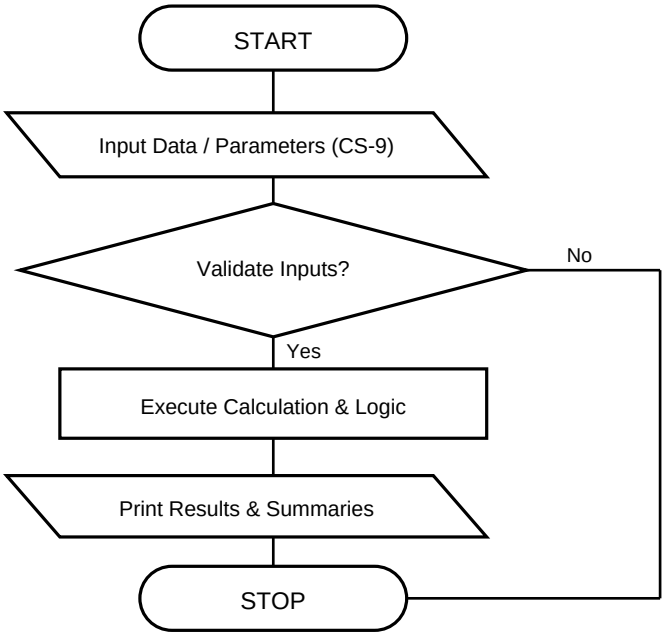
Algorithm

1. Read USD expenses and INR budget.
2. Validate inputs.
3. Converted INR = usd * 83.
4. Fee = 0.02 * converted INR.
5. Total INR = converted INR + fee.
6. Compare against budget.

Pseudocode

```
START
INPUT usd_expense, budget_inr
IF usd_expense < 0 OR budget_inr < 0 THEN
    PRINT "Invalid input"
ELSE
    inr_converted = usd_expense * 83
    fee = inr_converted * 0.02
    total_inr = inr_converted + fee
    PRINT total_inr
    IF total_inr <= budget_inr THEN PRINT "Within Budget" ELSE PRINT "Over Bu
dget"
END IF
STOP
```

Flowchart



Source Code

```
usd_expense = float(input("Enter travel expenses in USD: "))
budget_inr = float(input("Enter allocated budget in INR: "))

if usd_expense < 0 or budget_inr < 0:
    print("Invalid input.")
else:
    converted = usd_expense * 83.0
    fee = converted * 0.02
    total_inr = converted + fee
    print("Converted Amount: Rs.", round(converted, 2))
    print("Conversion Fee (2%): Rs.", round(fee, 2))
    print("Total Expense in INR: Rs.", round(total_inr, 2))
    if total_inr <= budget_inr:
        print("Status: Within Budget")
    else:
        print("Status: Over Budget")
```

Test Cases

1. USD: 500, Budget: 50000 -> Total INR: 42330.0 -> Status: Within Budget
2. USD: 700, Budget: 50000 -> Total INR: 59262.0 -> Status: Over Budget

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\9.py
Enter travel expenses in USD: 500
Enter allocated budget in INR: 50000
Converted Amount: Rs. 41500.0
Conversion Fee (2%): Rs. 830.0
Total Expense in INR: Rs. 42330.0
Status: Within Budget

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\9.py
Enter travel expenses in USD: 700
Enter allocated budget in INR: 50000
Converted Amount: Rs. 58100.0
Conversion Fee (2%): Rs. 1162.0
Total Expense in INR: Rs. 59262.0
```

```
Status: Over Budget

Process finished with exit code 0
```

Conclusion

The multi-currency travel budget system converts foreign currency and monitors overall budget compliance.

10.

Python Activity-Based Learning

Campus Event Financial Planning: Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

Sol:

Problem statement

Calculate total campus event execution costs and determine minimum participant registrations required to break even.

Objectives

Aggregate venue, catering, and equipment costs.

Determine registration fee income.

Calculate break-even participant count.

Assumptions

Fixed Event Cost = Venue + Equipment.

Variable Cost per Participant = Catering + Certificate.

Registration Fee per Participant is specified.

Algorithm

1. Read fixed cost, variable cost per head, and registration fee per head.

2. Validate inputs (fee > variable cost).

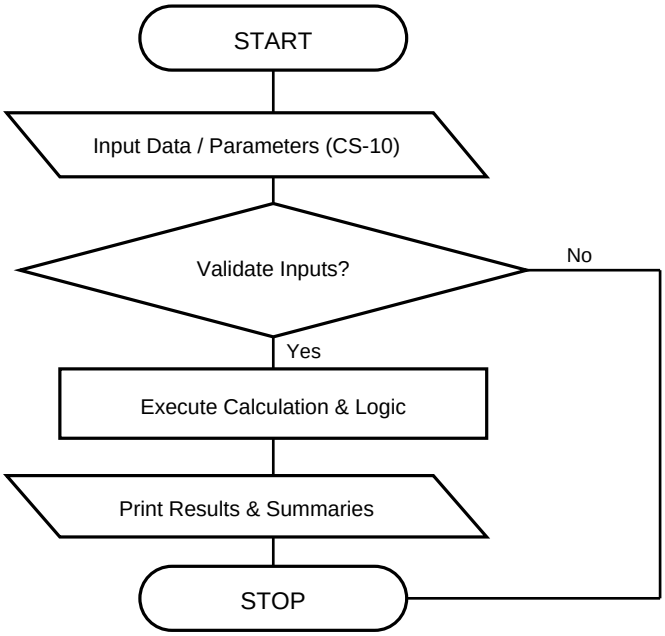
3. Break-even count = fixed cost / (fee - variable cost).

4. Display break-even registrations required.

Pseudocode

```
START
INPUT fixed_cost, var_cost, reg_fee
IF fixed_cost < 0 OR var_cost < 0 OR reg_fee <= var_cost THEN
    PRINT "Invalid input"
ELSE
    break_even = CEIL(fixed_cost / (reg_fee - var_cost))
    PRINT break_even
END IF
STOP
```

Flowchart



Source Code

```
import math

fixed_cost = float(input("Enter fixed event cost (Venue+Equipment): "))
var_cost = float(input("Enter variable cost per participant: "))
reg_fee = float(input("Enter registration fee per participant: "))

if fixed_cost < 0 or var_cost < 0 or reg_fee <= var_cost:
    print("Invalid financial parameters.")
else:
    margin = reg_fee - var_cost
    break_even = math.ceil(fixed_cost / margin)
    print("Contribution Margin per Head: Rs.", round(margin, 2))
    print("Minimum Registrations to Break Even:", break_even)
```

Test Cases

1. Fixed: 10000, Var Cost: 100, Fee: 300 -> Margin: 200 -> Break-even: 50 participants
2. Fixed: 15000, Var Cost: 150, Fee: 250 -> Margin: 100 -> Break-even: 150 participants

Output

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\10.py
Enter fixed event cost (Venue+Equipment): 10000
Enter variable cost per participant: 100
Enter registration fee per participant: 300
Contribution Margin per Head: Rs. 200.0
Minimum Registrations to Break Even: 50

Process finished with exit code 0
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\10.py
Enter fixed event cost (Venue+Equipment): 15000
Enter variable cost per participant: 150
Enter registration fee per participant: 250
Contribution Margin per Head: Rs. 100.0
Minimum Registrations to Break Even: 150

Process finished with exit code 0
```

Conclusion

The financial planning model computes event overheads and determines break-even attendance thresholds.